

# SIMATS ENGINEERING

## Assessment -CO1-AT2

---

*CO1: Apply AI basics such as intelligent agents and*

---

*uninformed search methods to solve problems effectively. (BL3)*

|                 |                         |
|-----------------|-------------------------|
| Subject         | Artificial Intelligence |
| Course Code     | CSA 1712                |
| Name            | Vignesh Kumar K         |
| Register Number | 192512375               |

### Declaration

*I, Vignesh Kumar K (Reg No: 192512375), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

# Question 1 — Constraint Satisfaction: Doctor Shift Scheduling

**Problem:** Assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) satisfying all given constraints.

## 1. CSP Formulation

**Variables:** D1, D2, D3

**Domain (each variable):** {Morning, Afternoon, Night}

**Constraints:**

- C1: D1 ≠ Night
- C2: D2 comes before D3 in the ordering Morning < Afternoon < Night
- C3: All-different — only one doctor per shift (each shift used exactly once)
- C4: D3 ≠ Morning
- C5: Every doctor must be assigned exactly one shift (implied by C3, a complete bijection)

## 2. Backtracking Search — Step-by-Step Trace

Variable order used: D1 → D2 → D3. Value order tried at each step: Morning → Afternoon → Night.

| Step | Variable | Value Tried | Constraint Check                                                                               | Result                                 |
|------|----------|-------------|------------------------------------------------------------------------------------------------|----------------------------------------|
| 1    | D1       | Morning     | C1 (D1 ≠ Night): satisfied                                                                     | Accepted — move to D2                  |
| 2    | D2       | Morning     | C3 (all-different): conflicts with D1 = Morning                                                | Rejected — try next value              |
| 3    | D2       | Afternoon   | C3: no conflict with D1                                                                        | Accepted — move to D3                  |
| 4    | D3       | Morning     | C4 (D3 ≠ Morning): violated                                                                    | Rejected — try next value              |
| 5    | D3       | Afternoon   | C3: conflicts with D2 = Afternoon                                                              | Rejected — try next value              |
| 6    | D3       | Night       | C3: no conflict · C4: Night ≠ Morning ✓ · C2: Afternoon < Night ✓ · C1: D1 = Morning ≠ Night ✓ | All constraints satisfied — COMPLETE ✓ |

## 3. Final Valid Schedule

| Doctor | Assigned Shift |
|--------|----------------|
| D1     | Morning        |
| D2     | Afternoon      |
| D3     | Night          |

#### 4. Verification

- C1 — D1 ≠ Night: D1 = Morning ✓
- C2 — D2 before D3: Afternoon comes before Night ✓
- C3 — Only one doctor per shift: {Morning, Afternoon, Night} all used exactly once ✓
- C4 — D3 ≠ Morning: D3 = Night ✓
- C5 — Every doctor assigned exactly one shift ✓

All five constraints are satisfied, confirming **D1 = Morning, D2 = Afternoon, D3 = Night** is a valid solution.

Backtracking search reached it after two rejected branches (Step 2 and the Step 4–5 dead end under D3), demonstrating how the algorithm prunes inconsistent partial assignments and backtracks until a fully consistent assignment is found.

#### Question 2 — Robot Navigation in a 5×5 Grid using BFS

---

No specific obstacle layout was provided with the question, so the following representative 5×5 grid is assumed to demonstrate the method (rows and columns numbered 1–5, top-left to bottom-right). **S** = Start (1,1), **G** = Goal (5,5), ■ = obstacle.

|   |   |   |   |   |
|---|---|---|---|---|
| S | . | . | . | . |
| . | ■ | . | ■ | . |
| . | . | . | ■ | . |
| . | ■ | . | . | . |
| . | . | . | . | G |

*Yellow = Start/Goal · Grey = obstacle · Green = the optimal path found below.*

##### 1. Problem Setup

**Start (S):** (1,1) **Goal (G):** (5,5)

**Obstacles:** (2,2), (2,4), (3,4), (4,2)

**Actions:** Up, Down, Left, Right — each with cost = 1. Manhattan distance  $h(n) = |r - 5| + |c - 5|$  is shown alongside each expansion for reference (BFS itself expands purely in FIFO order and does not use  $h(n)$  to choose which node to expand next, but the heuristic value is reported here to show how the estimated remaining distance shrinks as the search progresses).

##### 2. Step-by-Step Node Expansion (FIFO Queue)

| Iter. | Node Expanded | $h(n)$ | Neighbours Generated                  | Queue After             |
|-------|---------------|--------|---------------------------------------|-------------------------|
| 1     | (1,1)         | 8      | Down→(2,1), Right→(1,2)               | (2,1),(1,2)             |
| 2     | (2,1)         | 7      | Down→(3,1)                            | (1,2),(3,1)             |
| 3     | (1,2)         | 7      | Right→(1,3)                           | (3,1),(1,3)             |
| 4     | (3,1)         | 6      | Down→(4,1), Right→(3,2)               | (1,3),(4,1),(3,2)       |
| 5     | (1,3)         | 6      | Down→(2,3), Right→(1,4)               | (4,1),(3,2),(2,3),(1,4) |
| 6     | (4,1)         | 5      | Down→(5,1)                            | (3,2),(2,3),(1,4),(5,1) |
| 7     | (3,2)         | 5      | Right→(3,3)                           | (2,3),(1,4),(5,1),(3,3) |
| 8     | (2,3)         | 5      | (no new — neighbours visited/blocked) | (1,4),(5,1),(3,3)       |
| 9     | (1,4)         | 5      | Right→(1,5)                           | (5,1),(3,3),(1,5)       |
| 10    | (5,1)         | 4      | Right→(5,2)                           | (3,3),(1,5),(5,2)       |
| 11    | (3,3)         | 4      | Down→(4,3)                            | (1,5),(5,2),(4,3)       |
| 12    | (1,5)         | 4      | Down→(2,5)                            | (5,2),(4,3),(2,5)       |
| 13    | (5,2)         | 3      | Right→(5,3)                           | (4,3),(2,5),(5,3)       |
| 14    | (4,3)         | 3      | Right→(4,4)                           | (2,5),(5,3),(4,4)       |
| 15    | (2,5)         | 3      | Down→(3,5)                            | (5,3),(4,4),(3,5)       |
| 16    | (5,3)         | 2      | Right→(5,4)                           | (4,4),(3,5),(5,4)       |
| 17    | (4,4)         | 2      | Right→(4,5)                           | (3,5),(5,4),(4,5)       |
| 18    | (3,5)         | 2      | (no new — neighbours visited/blocked) | (5,4),(4,5)             |
| 19    | (5,4)         | 1      | Right→(5,5) = GOAL                    | (4,5),(5,5)             |
| 20    | (4,5)         | 1      | (no new)                              | (5,5)                   |
| 21    | (5,5)         | 0      | GOAL REACHED — stop                   | —                       |

### 3. Optimal Path and Cost

Tracing back parent pointers from the goal:

**Optimal path:** (1,1) → (2,1) → (3,1) → (4,1) → (5,1) → (5,2) → (5,3) → (5,4) → (5,5)

**Total cost:** 8 moves × 1 = 8

This equals the Manhattan-distance lower bound  $h(S) = |1-5| + |1-5| = 8$ , confirming the path BFS found is indeed the shortest possible route around the obstacles — BFS is guaranteed optimal here because every move has equal (unit) cost.

## Question 3 — Autonomous Rescue Robot: Search Under Partial Observability

The robot starts at S in a grid of rooms and must reach a survivor at G, where some cells are blocked, only adjacent cells can be sensed, movement costs 1 (plus an extra 2 for risky cells), and the robot must update its knowledge as it moves (an online search scenario).

### (a) How the Constraints Affect the Robot's Decision-Making

| Constraint                                          | Effect on Decision-Making                                                                                                                                                              |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Movement restricted to Up/Down/Left/Right           | Fixes the action set and branching factor ( $\leq 4$ successors per state), defining the transition model the robot reasons over at each step.                                         |
| Some cells blocked (obstacles)                      | Removes certain successor states dynamically; the robot must treat a sensed obstacle as an invalid action and choose an alternative direction.                                         |
| Limited sensing (adjacent cells only)               | Makes the environment partially observable — the robot cannot plan a complete optimal route in advance; it must explore incrementally and revise its plan as new cells become visible. |
| Move cost 1, risky-zone cost +2 (total 3)           | Turns the problem into a weighted-cost search rather than a simple step-counting search, so path selection must weigh safety against distance, favouring cost, not just hop count.     |
| Must minimize total cost while staying safe         | Defines the performance measure / objective function the robot optimizes — cheapest-safe-path, not merely shortest-path.                                                               |
| Must update knowledge dynamically (online scenario) | Rules out classical "offline" search that assumes the whole map is known beforehand; requires an online search agent that interleaves sensing, planning, and acting.                   |

### (b) Solution Approach — Online Uniform Cost Search

Because the map is only partially known and revealed as the robot moves, a plain offline UCS (which assumes the full graph is known in advance) cannot be applied directly. The most suitable approach is an **Online Search Agent using Uniform-Cost principles** (an online / real-time cost-based search, in the style of Online-DFS-Agent or LRTA\*, adapted to expand by cumulative cost rather than plain depth):

- 1. Initialize: current state = S, cumulative cost  $g = 0$ , and an internal knowledge map (initially empty except S).
- 2. Sense: observe the status (free / blocked / risky) of all currently adjacent cells.
- 3. Update knowledge base with the newly sensed information.
- 4. Evaluate candidates: for each unvisited, non-blocked neighbour, compute the cost of moving there — 1 for a normal cell,  $1 + 2 = 3$  for a risky cell.
- 5. Select action: move to the unvisited neighbour with the lowest cumulative cost  $g(n)$ , consistent with the UCS expansion rule, but decided locally since the full graph is unknown.
- 6. Act and update: physically move, add the incurred cost to  $g$ , mark the cell visited.

- 7. Backtrack when stuck: if all neighbours of the current cell are blocked, visited, or dead-ended, backtrack to the most recent cell that still has an unexplored option (as in Online DFS backtracking).
- 8. Repeat steps 2–7 until the current state equals G.
- 9. Return the path actually traversed and its total accumulated cost.

### (c) Demonstrating AI Concepts

| Concept           | How It Applies Here                                                                                                                                                                                                                                                                                                                                             |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Intelligent Agent | The robot follows the classic percept–action loop: it perceives its adjacent cells (sensors), maintains an internal knowledge map, and chooses movement actions (actuators) to progress toward rescuing the survivor — a textbook example of an agent function mapping percept sequences to actions.                                                            |
| Rationality       | At every step the robot is rational because, given only the percepts and knowledge available to it so far, it always chooses the action expected to minimize cumulative cost while making safe progress toward G — it does the best it can with the information available, which is the definition of bounded/limited-information rationality, not omniscience. |
| Environment Type  | Partially observable (only adjacent cells sensed), dynamic/unknown (knowledge is built incrementally as it moves), sequential (each move affects future options and costs), discrete (grid-based states and actions), and single-agent.                                                                                                                         |

### (d) Illustrative Cost Calculation

Suppose the robot's actual traversed path (discovered online) covers 7 normal cells and 1 risky cell before reaching G:

Total cost =  $(7 \text{ moves} \times 1) + (1 \text{ risky move} \times 3) = 7 + 3 = \mathbf{10}$  — illustrating how the robot's online cost accumulation naturally penalises routes through risky zones, steering it toward safer cells whenever an equal- or lower-cost safe alternative exists.

### (e) Justification of the Chosen Approach

- Partial observability rules out a classical, fully-planned search — the robot genuinely does not know the whole map, so it must sense-plan-act incrementally, which is exactly what an online search agent is designed for.
- The non-uniform cost structure (normal vs. risky cells) means a plain BFS/DFS that only counts steps would be inadequate — cost-based (UCS-style) evaluation is required to guarantee the cheapest-safe path rather than merely the shortest one.
- The requirement to "update knowledge dynamically" directly matches the defining behaviour of online search agents, which revise their internal map and plan as new percepts arrive, unlike offline algorithms that assume a static, fully-known environment.
- Together, an online, cost-sensitive search strategy best satisfies all the stated constraints simultaneously: safety, dynamic knowledge, and minimum total cost.